# ECG Signal Generator

## Generating Synthetic ECG Signals Using a BiLSTM-CNN GAN

CE903 Group Project → Team 8

Khalifa Alkuwari
Noureldin Abdelhafez
Aly Khalil
Hassan Mohamed
Omar Khalel
Moustafa AbouElkheir
Malak Sheat
Zeyad Abouelrous
Negin Mashayekhi

Supervisor: Dr Delaram Jarchi

University of Essex: School of Computer Science and Electronic Engineering

# Dataset

PhysioNet/CinC Challenge 2017 [2]

## Dataset Overview

- PhysioNet/CinC Challenge 2017
- 8,528 single-lead ECG recordings
- Each recording: 30–60 seconds long
- Sampling rate: 300 Hz
- Most recordings ~9,000 samples
- Format: .mat files + REFERENCE.csv

## Class Distribution

**8,528 total recordings**

↓

**Normal (N): 5,050 — 59.2%**

**AF (A): 738 — 8.7%**

**Other (O): 2,456 — 28.8%**

**Noisy (~): 284 — EXCLUDED**

↓

**8,244 recordings used**

## Why This Dataset?

- Supervisor directed use of PhysioNet 2017
- Much larger than MIT-BIH (8,528 vs 48 recordings) [1]
- Explicit class labels enable multi-class generation
- Publicly available so no patient privacy concerns
- Single-lead recordings match the paper's approach [1]

# Data Preprocessing

Transforming raw ECG recordings into GAN-ready training data

## Pipeline

1. Load 8,244 ECG recordings (.mat files) from PhysioNet 2017 dataset. [2]
2. Bandpass filter (0.5–40 Hz) removes slow baseline drift from breathing, electrical powerline interference, and high-frequency muscle artifacts

3. Min-max normalisation scales each recording independently to [0, 1]
4. Segment each recording into 3,000-sample windows with no overlap. So a 30-second recording with 9,000 samples gives us three 10-second segments

**Key Decisions**:

- Split 80/20 by recording ID before segmentation to prevent data leakage
- Segment length 3,000 (divides evenly into 30s recordings, 3 segments each)
- Noisy class excluded — corrupted data, not a heart condition

## Output

**8,528 total recordings**
**Exclude Noisy class**

↓

**8,244 recordings (N: 5,050 / A: 738 / O: 2,456)**
**Preprocess + Segment**

↓

**26,319 total segments**
**80/20 split by recording ID**

↓

**Train: 21,021 segments (ecg_train.npy)**

↓

**Test: 5,298 segments (ecg_test.npy)**

### Deviations from Paper [1]

· Dataset: PhysioNet 2017 instead of MIT-BIH

· Normalisation: min-max [0,1] instead of divide by 200 (different data encoding)

- Bandpass filter: added (not in paper) — PhysioNet data noisier than MIT-BIH

· Segment length: 3,000 instead of 3,120

# ECG GAN Discriminator

## Final Architecture

PhysioNet 2017 Challenge · CE903

**2**
Conv Layers

**10**
Max Channels

**3000**
Input Points

**45**
FC Inputs

ARCHITECTURE

# Architecture Pipeline

3000-point ECG signal compressed through two conv+pool blocks to 45 features, classified in two FC layers

| INPUT | CONV 1 | POOL 1 | CONV 2 | POOL 2 | FLATTEN | FC 1 | OUTPUT |
|---|---|---|---|---|---|---|---|
| (B, 1, 3000) | k=120 s=5 | k=46 s=3 | k=36 s=3 | k=24 s=3 | 5 × 9 = 45 | 45 → 25 | 25 → 1<br>Sigmoid |
| ch 1 | ch 10 | ch 10 | ch 5 | ch 5 | — | — | — |
| T 3000 | T 577 | T 178 | T 48 | T 9 | T 45 | 25 | P(real) |

Input · Conv1d · MaxPool1d · Flatten · Linear · Output

# Convolutional Layers

Two Conv1d blocks, each followed by MaxPool1d

## Conv Layer 1

nn.Conv1d(1, 10, kernel_size=120, stride=5)

| | |
|---|---|
| in_channels | 1 |
| out_channels | 10 |
| kernel_size | 120 → 400 ms @ 300 Hz |
| stride | 5 → T: 3000 → 577 |
| MaxPool1d | k=46, s=3 → T: 577 → 178 |

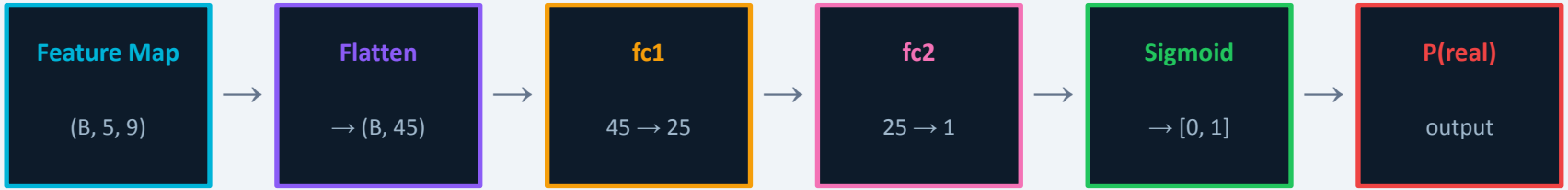
## Conv Layer 2

nn.Conv1d(10, 5, kernel_size=36, stride=3)

| | |
|---|---|
| in_channels | 10 |
| out_channels | 5 (channels reduce) |
| kernel_size | 36 → 120 ms @ 300 Hz |
| stride | 3 → T: 178 → 48 |
| MaxPool1d | k=24, s=3 → T: 48 → 9 |

# Classification Head

5 × 9 = 45 features ⟶ two linear layers ⟶ P(real)




### Why only 45 inputs?

Two rounds of Conv+Pool compress 3000 timesteps down to 9. With 5 channels that is 45 values — the entire classification decision is made from this small representation.

### Two FC layers (45⟶25⟶1)

A hidden layer of 25 neurons adds one non-linear step before the final score. Total FC parameters: (45×25 + 25) + (25×1 + 1) = 1,176.

### Sigmoid output

Squashes the score to [0, 1], readable as P(real).

# GAN Generator

## Architecture & Design

**2**

BiLSTM Layers

**1**

Dropout + FC + Sigmoid

**(bach, seq_len, noise_dim)**

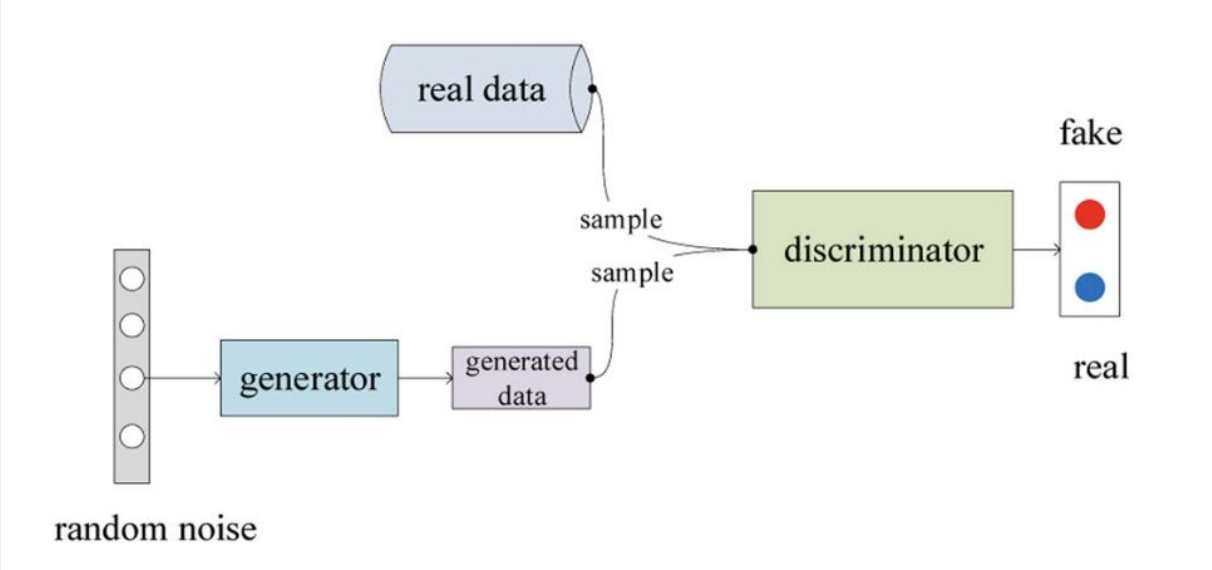
Input Dim

**(batch, seq_len, 1)**

Output Dim

# Generator Role

- Generator uses Gaussian **noise** as input.
- It creates synthetic ECG signals.
- Continuous **feedback** between generator and discriminator.
- Generator keeps **improving** as discriminator distinguishes.



# Input & Output of Generator

## Input

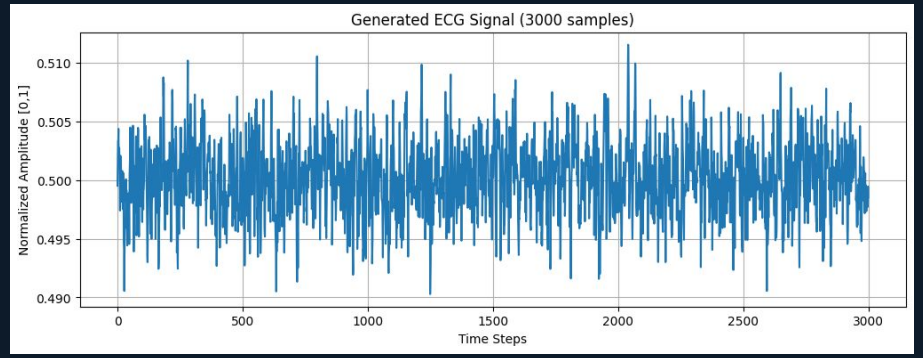
**Random Noise**

- Shape: (batch_size, seq_len, noise_dim)
  - Seq_len = 3000
  - Noise_dim = 5
- Input noise follows Gaussian distribution

## Output

**Generated ECG Signal**

- Shape: (batch_size, seq_len, 1)
- Generates ECG sequence over time
  - Each timestep = 1 value

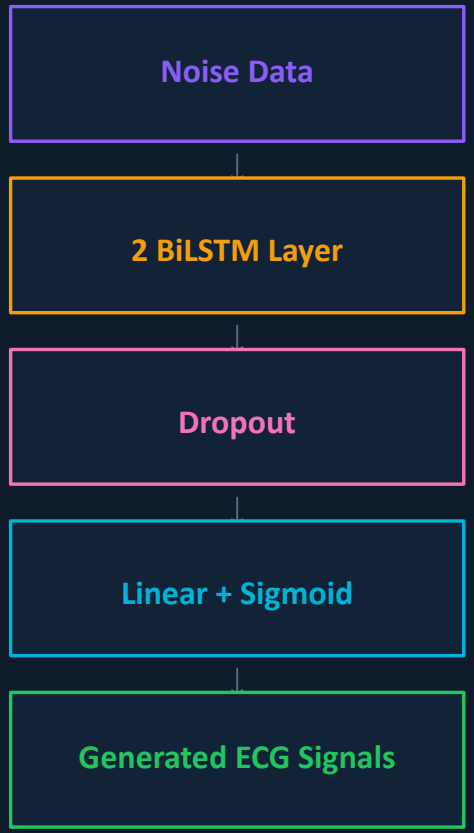


# Architecture Pipeline

| INPUT | BiLSTM 1 | BiLSTM 2 | Dropout | Linear | Sigmoid |
|---|---|---|---|---|---|
| seq_len | input_size | input_size | p | input_size | Output Range |
| 3000 | 5 | 512 | 0.3 | 512 | [0,1] |
| noise_dim | hidden_size | hidden_size | | neurons | |
| 5 | 256 | 256 | | 1 | |

Input · Layers · Activation Function

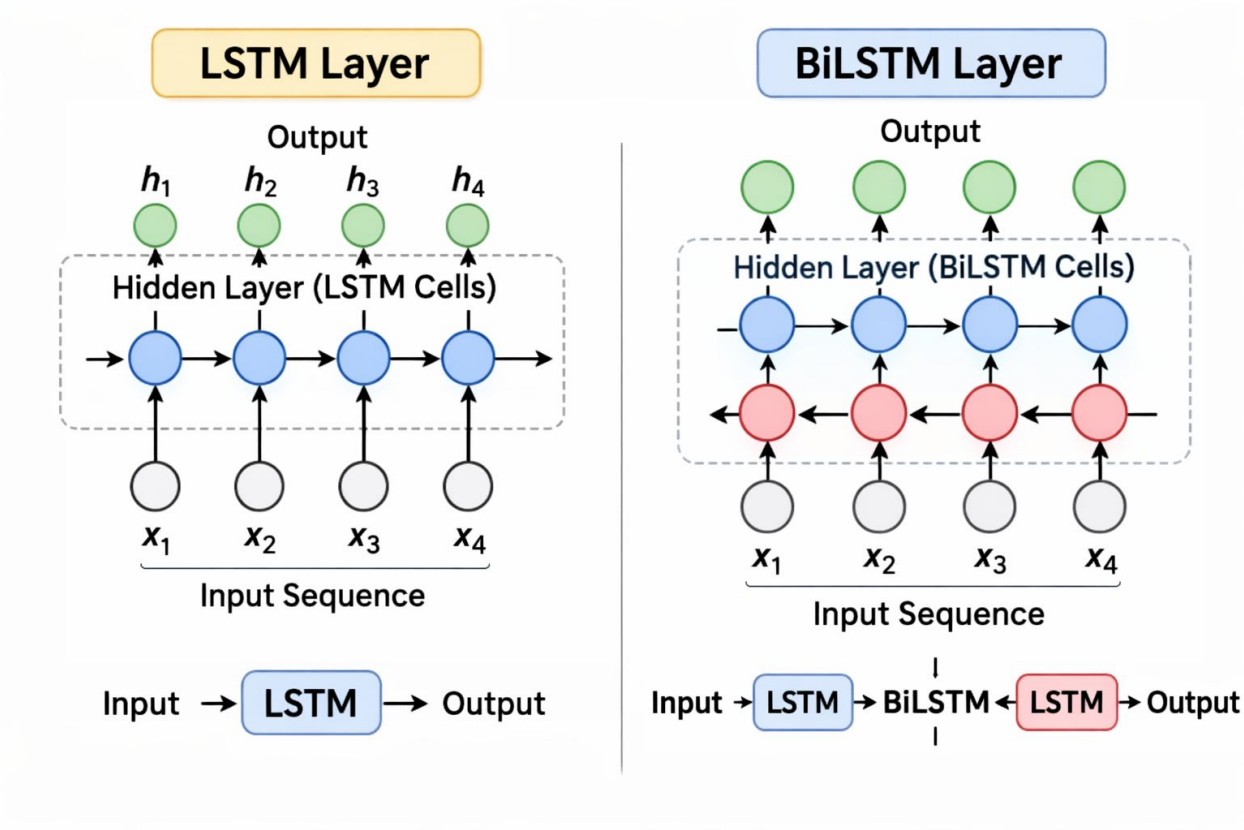
# Forward




1. The generator starts with **random noise as input**.
2. This noise is passed through **two BiLSTM layers** to learn temporal patterns.
3. Then we apply **dropout** to reduce overfitting.
4. The **linear layer** maps features to a single ECG value per timestep.
5. Finally, **sigmoid** ensures the output stays in a valid range.
6. The result is a **synthetic ECG signal** generated from noise.

# Why BiLSTM Layer?

- LSTM only learns from **previous time steps**
- BiLSTM captures **both past and future context**
- Provides **richer context at each time step**
- Important for ECG since patterns depend on **before and after signals**
- Well-suited for **time-series sequence modeling**



# GAN Training

## Objective, Loop & Key Findings

Stable adversarial balance mattered more than simply training longer.

**BCE**
Loss Function

**Adam**
Optimiser

**50**
Epochs Tested

**2**
Configs Tested

**Balance**
Main Finding

# Training: Adversarial Update Steps

Each batch alternates between improving the discriminator and improving the generator.

## Discriminator step (D)

1. Sample a batch of real ECG segments from the training set
2. Generate a Synthetic ECG batch from random noise
3. Apply instance noise to both real and fake inputs
4. Use smoothed labels instead of 1/0 targets
5. Update D to distinguish real from generated ECG signals

## Generator step (G)

1. Sample fresh random noise z
2. Generate synthetic ECG signals with G(z)
3. Pass them through D again with gradients preserved
4. Update G so that D is encouraged to classify fake signals as real
5. If D becomes too strong $d_real > 0.65$ and $d_fake_before < 0.20$, run an extra generator update to rebalance training

Stabilisation used throughout training: label smoothing, instance noise, and occasional extra generator updates when discriminator dominance appeared.

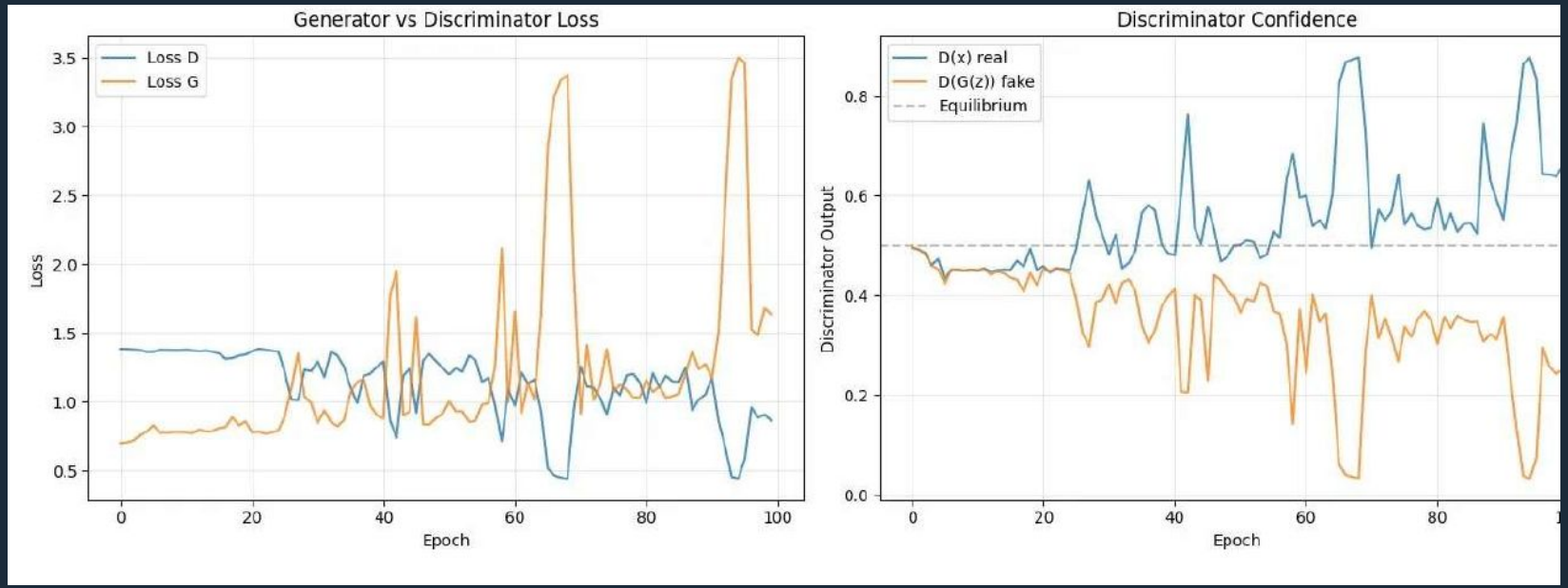
# Training Configurations Tested

Comparing training behaviour before and after adding label smoothing, instance noise, and extra generator updates

| Config | Key settings | Observed outcome |
|---|---|---|
| **Config 1** | • No label smoothing<br>• No instance noise<br>• No extra generator update | • D became too strong<br>• D overpowered G<br>• Training became unbalanced<br>• Generated ECG quality degraded |
| **Config 2** | • Added label smoothing<br>• Added instance noise<br>• Added extra generator update<br>• Extra $G$ step triggered when $d_real > 0.65$ and $d_fake_before < 0.20$ | • Better balance between $D$ and $G$<br>• Reduced discriminator dominance<br>• More stable training<br>• Improved generated ECG quality |

# Training Dynamics (50 epochs)

The curves indicate a more balanced training process, with lower discriminator dominance




Interpretation: when D(x) rises while D(G(z)) falls and generator loss spikes, the discriminator is dominating and the generator receives weaker learning signals.

# Evaluation

## Quantitative & Qualitative Checks

Held-out test data plus visual inspection were used to assess realism.

**RMSE**
0.298

**PRD**
63.89%

**FD**
0.358

**Visual**
Check

# Quantitative Evaluation

Computed on held-out ecg_test.npy; interpreted as approximate similarity indicators rather than exact one-to-one reconstructions.

## Metrics vs Acceptance Thresholds

| Metric | Value | Threshold | Status |
|---|---|---|---|
| RMSE | 0.298 | < 0.30 | PASS |
| PRD | 63.89% | < 70% | PASS |
| Frechet D. | 0.358 | < 0.85 | PASS |

## How we interpreted them

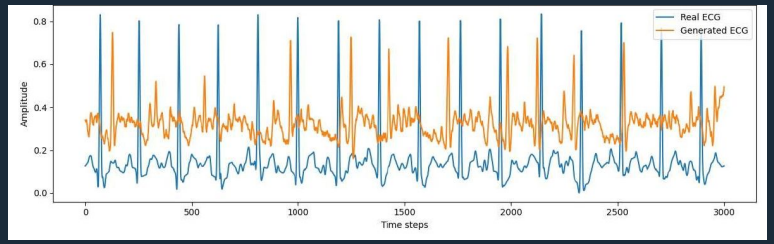
Generate fake samples to match the test-set size
RMSE and PRD measure overall error magnitude
Frechet distance measures shape similarity
Because the GAN is unconditional, fake sample i
is not meant to reproduce real sample i exactly
So these are approximate similarity indicators
and must be read with visual inspection

All three metrics passed the project thresholds, but visual assessment remained essential for judging ECG morphology quality.

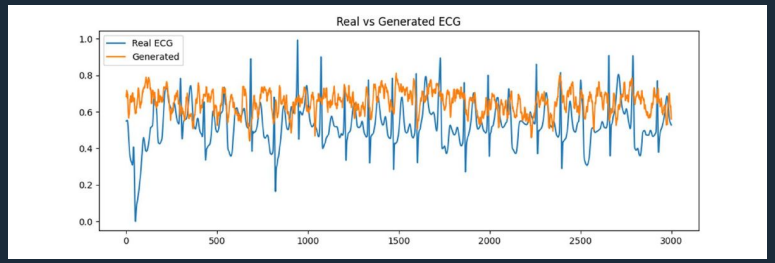
# Qualitative Results

Visual comparison of real vs generated ECG segments

## Best visual run (epoch 50)




## Evaluation checkpoint




Observation: the epoch-50 run shows sharper spike patterns and more recognisable ECG structure, while the evaluation checkpoint matches range better than detailed morphology.

# CardioGAN

## Synthesizing ECG from PPG Using Attentive GANs with Dual Discriminators

CE903 Group Project → Team 8

Khalifa Alkuwari
Noureldin Abdelhafez
Aly Khalil
Hassan Mohamed
Omar Khalel
Moustafa AbouElkheir
Malak Sheat
Zeyad Abouelrous
Negin Mashayekhi

Supervisor: Dr Delaram Jarchi

University of Essex: School of Computer Science and Electronic Engineering

# What is CardioGAN?

## The Problem

ECG captures critical cardiac data, but continuous wearable ECG is impractical. PPG sensors are ubiquitous (smartwatches) yet lack clinical precision and carry limited cardiac information.

## The Solution

CardioGAN learns to translate PPG ⟶ ECG using a CycleGAN backbone — no paired data required. An attention-based generator plus dual time/frequency discriminators produce clinically realistic ECG.

**9.74 ⟶ 2.89**

BPM Error
(PPG input ⟶ Generated ECG)

**125**

Participants
across 4 public datasets

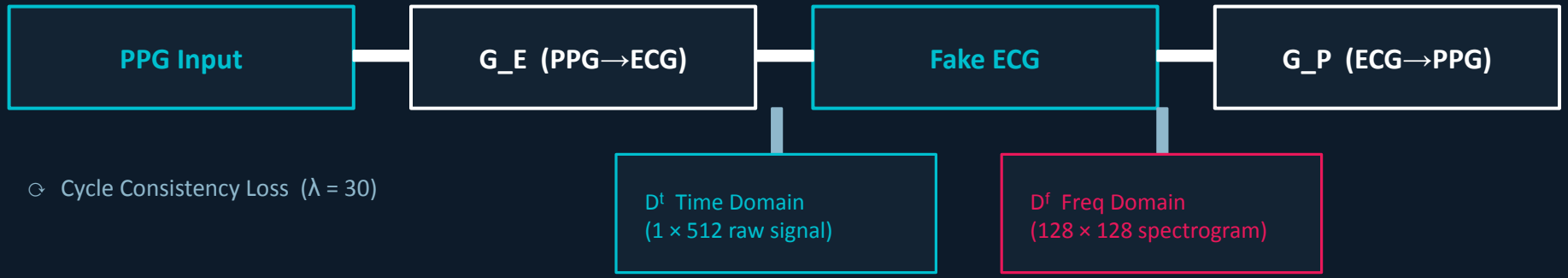
**~50 hrs**

Training Time
on NVIDIA Titan RTX

ARCHITECTURE

# Architecture

Attention U-Net generators with dual time and frequency discriminators




GENERATOR

## Attention U-Net

6-layer encoder/decoder · kernel 16, stride 2
Filters: 64→128→256→512→512→512
Self-gated soft-attention on all skip connections
Final output: tanh · Input: 1 × 512

TIME DISC

## $D^t$ PatchGAN

Input: (B, 1, 512) raw signal
4× Conv1D · [64,128,256,512] · k=16, s=2
LayerNorm (skip first layer)
Output: (B, 1, 16) · Weight α = 3

FREQ DISC

## $D^f$ PatchGAN

Input: (B, 1, 128, 128) log-STFT
4× Conv2D · [64,128,256,512] · k=7, s=2
LayerNorm (skip first layer)
Output: (B, 1, 4, 4) · Weight β = 1

# What We Built

### models.py

Attention U-Net Generator, TimeDisc, FreqDisc · differentiable torch.stft

### preprocessing.py

FIR/Butterworth filtering, resample to 128 Hz, z-score + min-max normalisation

### dataset.py

Unpaired CardioGANDataset with independent ECG / PPG shuffling

### train.py

Full CycleGAN loop · LR linear decay from epoch 10 · $\alpha=3$, $\beta=1$, $\lambda=30$

### inference.py

Batch PPG→ECG generation, overlap-add signal reconstruction + plots

### prepare_data.py

BIDMC, CAPNO, DaLiA, WESAD loaders + 80/20 subject-independent split

# Training Reality

**We completed the full PyTorch implementation — but were unable to train the model.**

The computational cost required was far beyond what was available to us within this project's timeframe.

GPU USED (PAPER)

## NVIDIA Titan RTX

24 GB VRAM

TRAINING TIME (PAPER)

## ~50 Hours

Continuous single-GPU compute

DATASET SCALE

## 58,000+

Segments · 125 subjects · 4 datasets

### What would be needed to run it:

- A high-memory GPU (≥ 16 GB VRAM) to accommodate the batch size of 128 used in the paper
- ~2–3 days of compute, or access to a cloud GPU (e.g. A100 / H100)
- All code is complete and ready — hardware availability is the only constraint

# References

[1] F. Zhu, F. Ye, Y. Fu, Q. Liu, and B. Shen, "Electrocardiogram generation with a bidirectional LSTM-CNN generative adversarial network," Scientific Reports, vol. 9, no. 1, pp. 1-11, 2019.

[2] PhysioNet/CinC Challenge 2017, "AF Classification from a Short Single Lead ECG Recording," https://physionet.org/content/challenge-2017/, 2017.

Thank you: we are happy to take questions